Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

Programming Fundamentals (Functions - II)

Last Lecture


```
#include <cmath>
#include <iostream>
using namespace std;

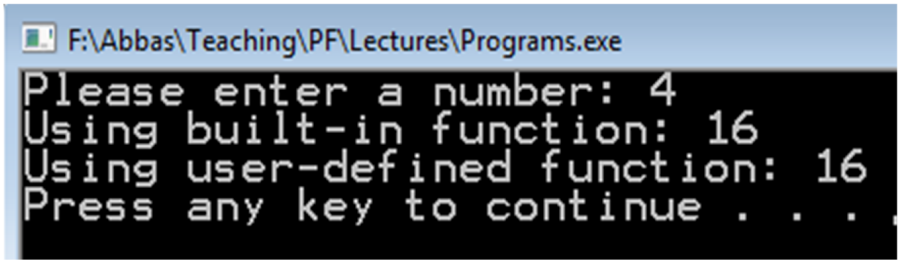
double square (double);
int main ()
   {

 double number;
 cout<<"Please enter a number: ";
 cin>>number;

 cout<<"Using built-in function: "<<pow(number, 2.0)<<endl;
 cout<<"Using user-defined function: "<<square(number)<<endl;

 return 0;
  }

double square (double n)
  {
   return n*n;
  }
```



Last Lecture

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

```
#include<iostream>
using namespace std;

void pass(int, int&);

int main()
  {
    int a = 5;
    int b = 5;
    cout<<" Value of a : "<<a;
    cout<<" and value of b : "<<b<<endl;
    pass(a,b);
    cout<<" Value of a : "<<a;
    cout<<" and value of b : "<<b<<endl;

 return 0;
}
 void pass(int a, int& b){a*=2; b*=2;}
```

```
E:\Abbas\Teaching\PF\Lectures\.
Value of a : 5 and value of b : 5
Value of a : 5 and value of b : 10
```

Last Lecture

Exercise:

Write the definition of a function that takes as input the three numbers. The function returns true if the first number to the power of the second number equals the third number; otherwise, it returns false. Assume that the three numbers are of type double.

In this Lecture

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

- **Inline functions**
  - How do we use *inline* functions?
  - Why do we use *inline* functions?
- **Default arguments**
  - Rules for defining the default arguments
- **Functions overloading**
  - Rules for function overloading
- **Templates**
  - Generic functions
- **Storage classes**
  - Visibility, lifetime and a location of a variable

Inline Functions

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

- While working with functions, each time a function is called, a significant amount of execution-time is spent by the calling and the return mechanism.
- Typically, arguments are pushed on to the stack and various registers are saved when a function is called, and then restored when the function returns.
- These instructions take a lot of time and is a overhead especially for small functions.
- C++ provides inline functions to avoid this overhead. An *inline* function is defined by the use of *inline* qualifier before a function's return type in function definition.

Inline Functions

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

- The general form for an inline function is:

```
inline returnType functionName(parameter – list){
    ...
    ...
    return value;
}
```

- With this definition, we are suggesting the compiler to replace calls with inline code.
- The compiler generates a copy of the function's code and replaces it with function call.
- Due to this, multiple copies of the same function code are inserted to replace multiple calls to the same function rather than having a single copy of the function to which control is passed each time the function is called.

Inline Functions

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

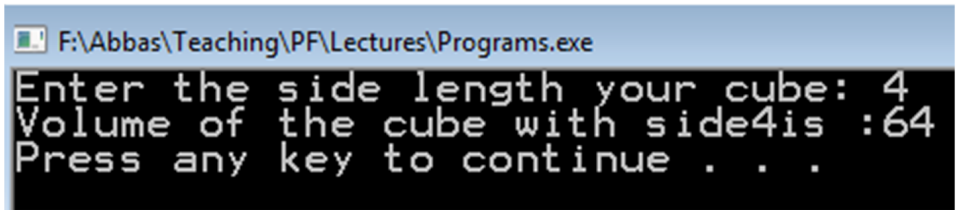
Conclusions

- This increases the program size. That is why *inline* qualifier should be used only with very small functions.
- Note that the *inline* is actually a request to the compiler and the compiler may ignore it due to some reasons. For example, a compiler usually does not *inline* a recursive function.
- When we declare a function as *inline*, the definition must be available in every source file that calls the function. For this reason, the definition of an *inline* function appears in a header file rather than a source file. This header file is included into each source file that uses the function.

Inline Functions


```
#include<iostream>
using namespace std;
inline double cube(double s)
      {
    return s*s*s;
        }
int main(){
   cout<<"Enter the side length your cube: ";
   double side;
   cin>>side;
   cout<<"Volume of the cube with side"<<side<<"is :"<<cube(side)<<endl;

   return 0;
}
```



Default Arguments

- C++ allows the programmer to specify default arguments and their default values. If a default argument is omitted in a call to a function, the default value of that argument is used.
- Default arguments must be the rightmost arguments in a function's parameter list. If an omitted argument is not the rightmost argument in the argument list, all the arguments to right of that argument also must be omitted.
- Default arguments should be specified with the first occurrence of the function name (typically in prototype).
- Default arguments also can be used with *inline* functions.

Default Arguments


```
#include<iostream>
using namespace std;
int volume (int length = 1,int width = 1,int height = 1); // default values
int main(){
   cout<<"The default volume is: "<<volume()<<endl
       <<"The volume with length 10 is: "<<volume(10)<<endl
       <<"The volume with length 10 and width 5 is: "<<volume(10, 5)<<endl
       <<"The volume with length 10, width 5 and height 2 is: "
       <<volume(10, 5, 2)<<endl;

    return 0;
  }

int volume(int  a, int  b, int  c){
    return a*b*c;
}
```

```
F:\Abbas\Teaching\PF\Lectures\Programs.exe
The default volume is: 1
The volume with length 10 is: 10
The volume with length 10 and width 5 is: 50
The volume with length 10, width 5 and height 2 is: 100
```

Default Arguments

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

Exercise: Which of the following are correct function prototypes?

1. $void\ display(int\ a = 1, int\ b = 1, int\ c = 1);$
2. $void\ display(int = 1, int = 1, int = 1);$
3. $void\ display(int = 1, int = 1, int);$
4. $void\ display(int = 1, int, int = 1);$
5. $void\ display(int, int = 1, int = 1);$

Exercise: Identify the correct Calls for the below prototype?

$void\ display(int, int = 1);$

1. $display();$
2. $display(5);$
3. $display(5, 10);$
4. $display(5, 10, 15);$

Functions Overloading

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

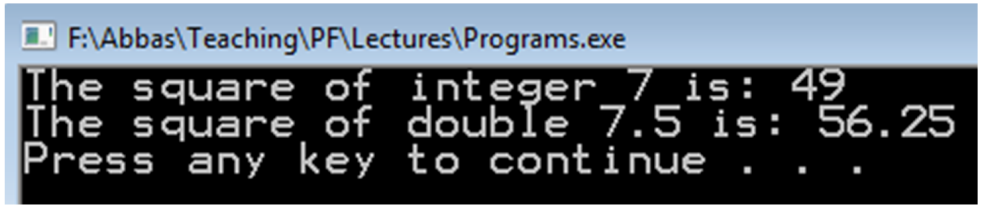
Conclusions

- C++ allows to define several functions of the same name but different sets of parameters. This capability is called function overloading.
- When an overloaded function is called, the C++ compiler selects the proper function by examining the number, types and order of the arguments in the call.
- Function overloading is a way through which C++ achieves polymorphism.
- Overloaded functions can have different return types but must have different parameters lists.
- The match between argument and parameter need not always be exact. In some case, C++ automatic type conversion can play a role to dissolve this situation.

Functions Overloading


```
#include<iostream>
using namespace std;
int square (int x){ return x*x; }
double square (double y){ return y*y;}
int main() {
      cout<<"The square of integer 7 is: "<<square(7)<<endl
          <<"The square of double 7.5 is: "<<square(7.5)<<endl;

   return 0;
  }
```



Functions Templates

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

- Overloaded functions are normally used to perform similar operations with different program logic on different data types.
- If the program logic and operations are identical for each data type then function templates are preferred.
- The function template starts with the keyword *template*. This is followed by a pair of angled brackets, which contains a list of parameters.
- Each parameter is preceded by either keyword *class* or keyword *typename*.
- The general format is:

template<*class type* > or *template* <*typename type* >

returnType functionName(*type P*1, *type P*2, ···)

{

...

return value;

}

Example: Conventional Method

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

```
#include<iostream>
using namespace std;

void swap_char(char&, char&);
void swap_int(int&, int&);
void swap_double(double&, double&);

int main()
 {
   char a = 'A'; char b = 'Z';
   cout<<"Characters before call: "<<a<<" and "<<b<<endl;
   swap_char(a,b) ;
   cout<<"Characters after call: "<<a<<" and "<<b<<endl;

   double x = 1.5; double y = 3.2;
   cout<<"Doubles before call: "<<x<<" and "<<y<<endl;
   swap_double(x,y);
   cout<<"Doubles after call: "<<x<<" and "<<y<<endl;

   int i = 2; int j = 5;
   cout<<"Integers before call: "<<i<<" and "<<j<<endl;
   swap_int(i,j);
   cout<<"Integers after call: "<<i<<" and "<<j<<endl;

 return 0;
 }
```

```
C:\Lectures\PF\Lecture.exe
Characters before call: A and Z
Characters after call: Z and A
Doubles before call: 1.5 and 3.2
Doubles after call: 3.2 and 1.5
Integers before call: 2 and 5
Integers after call: 5 and 2
```

Example: Conventional Method (2)

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

```
void swap_char(char& a, char& b)
    {
        char temp;
        temp = b;
        b = a;
        a = temp;
    }

void swap_double(double& a, double& b)
    {
        double temp;
        temp = b;
        b = a;
        a = temp;
    }

void swap_int(int& a, int& b)
    {
        int temp;
        temp = b;
        b = a;
        a = temp;
    }
```

Example: Function Overloading

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

```
#include<iostream>
using namespace std;

void swap(char&, char&);
void swap(int&, int&);
void swap(double&, double&);

int main()
 {
   char a = 'A'; char b = 'Z';
   cout<<"Characters before call: "<<a<<" and "<<b<<endl;
   swap(a,b);
   cout<<"Characters after call: "<<a<<" and "<<b<<endl;

   double x = 1.5; double y = 3.2;
   cout<<"Doubles before call: "<<x<<" and "<<y<<endl;
   swap(x,y);
   cout<<"Doubles after call: "<<x<<" and "<<y<<endl;

   int i = 2; int j = 5;
   cout<<"Integers before call: "<<i<<" and "<<j<<endl;
   swap(i,j);
   cout<<"Integers after call: "<<i<<" and "<<j<<endl;

 return 0;
 }
```

```
C:\Lectures\PF\Lecture.exe
Characters before call: A and Z
Characters after call: Z and A
Doubles before call: 1.5 and 3.2
Doubles after call: 3.2 and 1.5
Integers before call: 2 and 5
Integers after call: 5 and 2
```

Example: Function Overloading (2)


```
void swap(char& a, char& b)
    {
        char temp;
        temp = b;
        b = a;
        a = temp;
    }

void swap(double& a, double& b)
    {
        double temp;
        temp = b;
        b = a;
        a = temp;
    }

void swap(int& a, int& b)
    {
        int temp;
        temp = b;
        b = a;
        a = temp;
    }
```

Example: Function Template


```
#include<iostream>
using namespace std;

template <class T>
void Swap(T& a, T& b);

int main()
 {
   char a = 'A'; char b = 'Z';
   cout<<"Characters before call: "<<a<<" and "<<b<<endl;
   Swap(a,b);
   cout<<"Characters after call: "<<a<<" and "<<b<<endl;

   double x = 1.5; double y = 3.2;
   cout<<"Doubles before call: "<<x<<" and "<<y<<endl;
   Swap(x,y);
   cout<<"Doubles after call: "<<x<<" and "<<y<<endl;

   int i = 2; int j = 5;
   cout<<"Integers before call: "<<i<<" and "<<j<<endl;
   Swap(i,j);
   cout<<"Integers after call: "<<i<<" and "<<j<<endl;

 return 0;
 }

template <class T>
void Swap(T& a, T& b)
  {
      T temp = b;
      b = a; a = temp;
  }
```

```
C:\Lectures\PF\Lecture.exe
Characters before call: A and Z
Characters after call: Z and A
Doubles before call: 1.5 and 3.2
Doubles after call: 3.2 and 1.5
Integers before call: 2 and 5
Integers after call: 5 and 2
```

The *const* Qualifier

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

- We can define any kind of variable as "read only" variable by the use of *const* keyword.
- Constant variables must be initialized when they are declared. They cannot be modified thereafter. Any attempt to modify them will result in a syntax error.
- The general form to declare a constant variable is:

  const dataType varName = value;
- We can make the return type or arguments of a function as *const*. Value of a *const* argument cannot be modified. For built in data types, returning a *const* or non-*const* value doesn't make any difference.
- If a function has a non-*const* parameter, it can be passed a *const* as well as a non-*const* argument while making a call. Similarly, a function which has a *const* type parameter, can be passed a *const* type argument as well as a non-*const* argument.

Example 1

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

```
//void F(const int a){a = 100;} // invalid. 'a' can't be changed.
void F0(int a){a = 100; cout<<" I am F0.";}
void F1(int a){cout<<" I am F1.";}
void F2(const int a){cout<<" I am F2.";}
int main() {
   // const int a;  //'a' must be initialized when declared
   // a = 10; // Invalid.

   // const int a = 10; // ok
   //  a = 100; // invalid. 'a

  const int a = 10; int b = 5;
  //  F(a);  F(b);

    cout<<" \nF0() Called with const argument: "; F0(a);
    cout<<" \nF0() Called with non-const argument: ";F0(b);

    cout<<" \nF1() Called with const argument: "; F1(a);
    cout<<" \nF1() Called with non-const argument: ";F1(b);

    cout<<" \nF2() Called with const argument: "; F2(a);
    cout<<" \nF2() Called with non-const argument: ";F2(b);

return 0;
 }
```

```
E:\Abbas\Teaching\PF\Lectures\

F0() Called with const argument:  I am F0.
F0() Called with non-const argument:  I am F0.
F1() Called with const argument:  I am F1.
F1() Called with non-const argument:  I am F1.
F2() Called with const argument:  I am F2.
F2() Called with non-const argument:  I am F2.
```

Example 2


```
const int F1(const int a){return a;}

int main() {
   const int a = 10; int b = 5;
   cout<<"\n F1() Called with const argument: ";
   const int c = F1(a);
   //c++; Error! c can't be modified
   cout<<c;

   cout<<"\n F1() Called with const argument: ";
   int d = F1(a);
   d++;
   cout<<d;

   cout<<"\n F1() Called with non-const argument: ";
   const int e = F1(b);
   //e++; Error! e can't be modified
   cout<<e;

   cout<<"\n F1() Called with non-const argument: ";
   int f = F1(b);
   f++;
   cout<<f;

  return 0;
}
```

E:\Abbas\Teaching\PF\Lectures\

```
F1() Called with const argument: 10
F1() Called with const argument: 11
F1() Called with non-const argument: 5
F1() Called with non-const argument: 6
```

Example 3

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

```
const int F2(int a){return a;}

int main()
 {
   const int a = 10; int b = 5;
   cout<<"\n F2() Called with const argument: ";
   const int c = F2(a);
   //c++; Error! c can't be modified
   cout<<c;

   cout<<"\n F2() Called with const argument: ";
   int d = F2(a);
   d++;
   cout<<d;

   cout<<"\n F2() Called with non-const argument: ";
   const int e = F2(b);
   //e++; Error! e can't be modified
   cout<<e;

   cout<<"\n F2() Called with non-const argument: ";
   int f = F2(b);
   f++;
   cout<<f;

  return 0;
}
```

E:\Abbas\Teaching\PF\Lectures'

```
F2() Called with const argument: 10
F2() Called with const argument: 11
F2() Called with non-const argument: 5
F2() Called with non-const argument: 6
```

Example 4


```
int F3(const int a){return a;}

int main()
 {
   const int a = 10; int b = 5;
   cout<<"\n F3() Called with const argument: ";
   const int c = F3(a);
   //c++; Error! c can't be modified
   cout<<c;

   cout<<"\n F3() Called with const argument: ";
   int d = F3(a);
   d++;
   cout<<d;

   cout<<"\n F3() Called with non-const argument: ";
   const int e = F3(b);
   //e++; Error! e can't be modified
   cout<<e;

   cout<<"\n F3() Called with non-const argument: ";
   int f = F3(b);
   f++;
   cout<<f;

  return 0;
}
```

E:\Abbas\Teaching\PF\Lectures\

```
F3() Called with const argument: 10
F3() Called with const argument: 11
F3() Called with non-const argument: 5
F3() Called with non-const argument: 6
```

Example 5


```
int F4(int a){return a;}
int main()
 {
   const int a = 10; int b = 5;
   cout<<"\n F4() Called with const argument: ";
   const int c = F4(a);
   //c++; Error! c can't be modified
   cout<<c;

   cout<<"\n F4() Called with const argument: ";
   int d = F4(a);
   d++;
   cout<<d;

   cout<<"\n F4() Called with non-const argument: ";
   const int e = F4(b);
   //e++; Error! e can't be modified
   cout<<e;

   cout<<"\n F4() Called with non-const argument: ";
   int f = F4(b);
   f++;
   cout<<f;

 return 0;
}
```

E:\Abbas\Teaching\PF\Lectures\

```
F4() Called with const argument: 10
F4() Called with const argument: 11
F4() Called with non-const argument: 5
F4() Called with non-const argument: 6
```

Storage Classes

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

A storage class is used to describe the following information about a variable:

- The variable scope.
- The location where the variable will be stored.
- The initialized value of a variable.
- A lifetime of a variable.

Storage Classes

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

There are total four types of standard storage classes:

- *auto*: It is a default storage class. It is used for a local variable defined within a block or function.
- *static*: A local variable which is capable of retaining its value even when control is transferred to the function call.
- *register*: It is a variable which is stored inside a register.
- *extern*: It is a global variable. It is used for data sharing between C++ files.

The *auto* Storage Class

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

- The variables defined using *auto* storage class are called local variables.
- The word *auto* stands for automatic storage class.
- A variable is in *auto* storage class by default if it is not explicitly specified.
- The scope of an *auto* variable is limited with the particular block only. Once the control goes out of the block, the access is destroyed. Hence, only the block in which the *auto* variable is declared can access it.
- A keyword *auto* is used to define an *auto* storage class.
- By default, an *auto* variable contains a garbage value.

Variables defined inside a function without the *static* or *register* (or *extern*) keywords are *auto* variables. The storage class *auto* denies the possibility of the variable being global therefore it is impossible to create a global variable with the storage class *auto*.

The *auto* Storage Class


```
#include<iostream>
using namespace std;
void storage();
int main()
    {
     storage();
     storage();

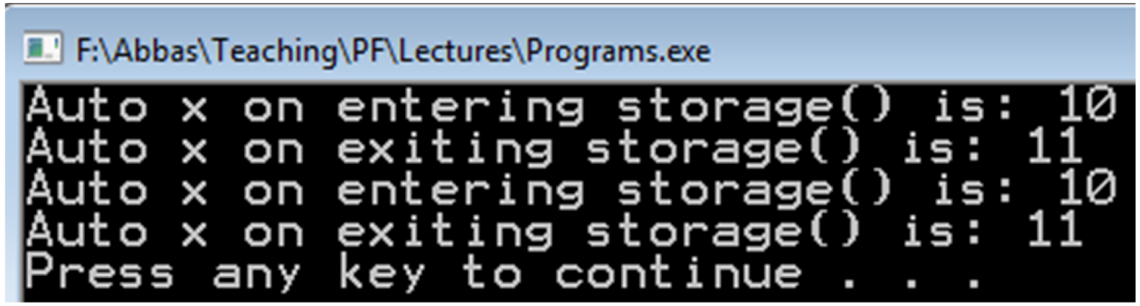
     return 0;
   }
void storage()
    {
     auto int x = 10;    // Local Variable
     cout<<"Auto x on entering storage() is: "<<x<<endl;
     ++x;
     cout<<"Auto x on exiting storage() is: "<<x<<endl;
   }
```

```
F:\Abbas\Teaching\PF\Lectures\Programs.exe
Auto x on entering storage() is: 10
Auto x on exiting storage() is: 11
Auto x on entering storage() is: 10
Auto x on exiting storage() is: 11
Press any key to continue . . .
```

The *auto* Storage Class


```
#include<iostream>
using namespace std;
void storage();
int main()
    {
     storage();
     storage();

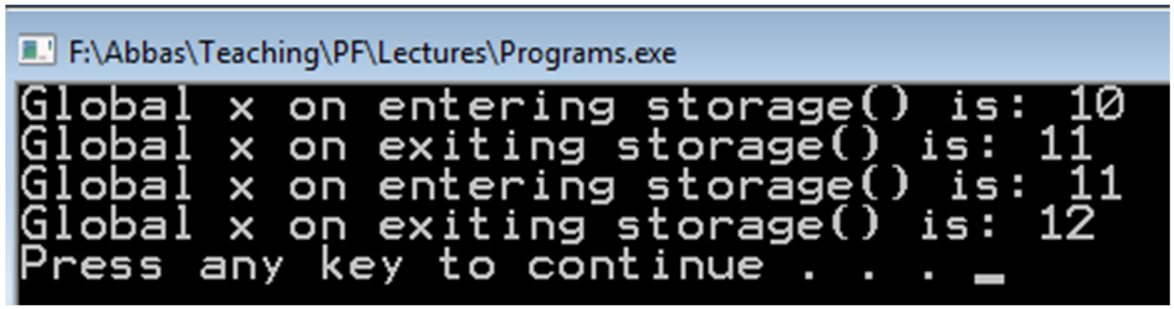
     return 0;
   }
void storage()
    {
     int x = 10;    // Local Variable
     cout<<"Auto x on entering storage() is: "<<x<<endl;
     ++x;
     cout<<"Auto x on exiting storage() is: "<<x<<endl;
   }
```



The *auto* Storage Class


```
#include<iostream>
using namespace std;
int x = 10;    // Global Variable (Not auto)
void storage();
int main()
    {
     storage();
     storage();

     return 0;
   }
void storage()
    {
     cout<<"Global x on entering storage() is: "<<x<<endl;
     ++x;
     cout<<"Global x on exiting storage() is: "<<x<<endl;
   }
```



The *auto* Storage Class

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

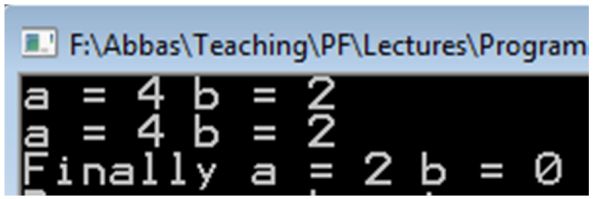
Storage Classes

Conclusions

```
#include<iostream>
using namespace std;

int a, b, c = 0;
void test ();
int main ()
    {
     int a = 1;
     test();
     a += 1;
     test();
     cout<<"Finally a = "<<a<<" b = "<<b<<endl;

     return 0;
    }
void test ()
    {
     int a = 2;
     int b = 1;
     a += ++b;
     cout<<"a = "<<a<<" b = "<<b<<endl;
     }
```



The *static* Storage Class

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

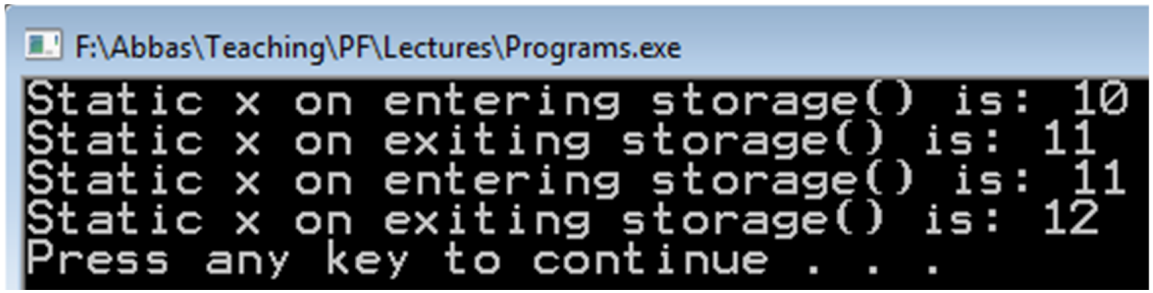
Conclusions

- The keyword *static* is used to declare a variable in *static* storage class.
- A *static* variable is initialized when declared.
- The *static* variables with a local scope are not destroyed when exiting the function. Therefore, a *static* variable retains its value forever and can be accessed when the function is re-entered.
- The *static* variables with a global scope are visible only to the file in which they are declared.
- In C++, *static* storage class cannot be used with function parameters.

The *static* Storage Class


```
#include<iostream>
using namespace std;
void storage();
int main()
    {
     storage();
     storage();

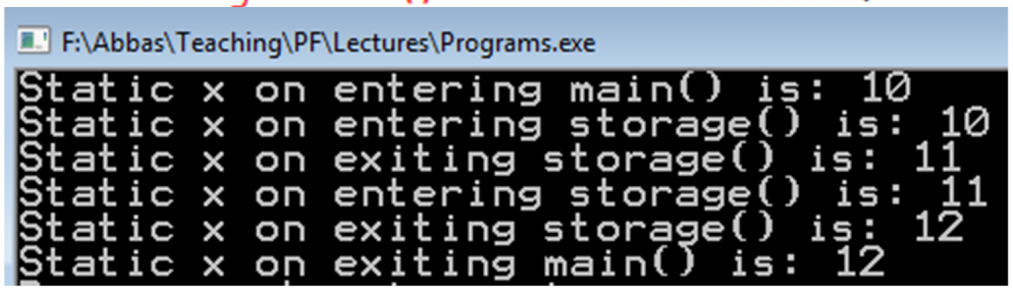
     return 0;
   }
void storage()
    {
     static int x = 10;    // Local Variable
     cout<<"Static x on entering storage() is: "<<x<<endl;
     ++x;
     cout<<"Static x on exiting storage() is: "<<x<<endl;
   }
```



The *static* Storage Class


```
#include<iostream>
using namespace std;
static int x = 10;    // Global Variable (static)
void storage();
int main()
    {
     cout<<"Static x on entering main() is: "<<x<<endl;
     storage();
     storage();
     cout<<"Static x on exiting main() is: "<<x<<endl;

     return 0;
   }
void storage()
    {
     cout<<"Static x on entering storage() is: "<<x<<endl;
     ++x;
     cout<<"Static x on exiting storage() is: "<<x<<endl;
   }
```



The *static* Storage Class

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

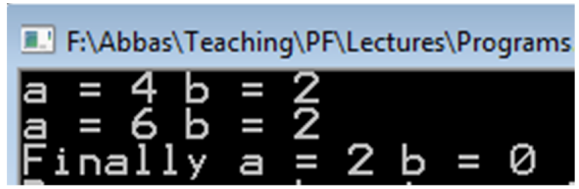
Storage Classes

Conclusions

```
#include<iostream>
using namespace std;

int a, b, c = 0;
void test ();
int main ()
    {
     static int a = 1;
     test();
     a += 1;
     test();
     cout<<"Finally a = "<<a<<" b = "<<b<<endl;

     return 0;
    }
void test ()
    {
     static int a = 2;
     int b = 1;
     a += ++b;
     cout<<"a = "<<a<<" b = "<<b<<endl;
     }
```



The *register* Storage Class

- The keyword *register* is used to declare a *register* storage class.
- It is similar to the *auto* storage class i.e. the variable is limited to the particular block. The only difference is that the variables declared using *register* storage class are stored inside CPU registers instead of a memory. Registers have faster access than that of the main memory.
- The variables declared using *register* storage class have no default value.
- These variables are often declared at the beginning of a program.

The *extern* storage class

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

- The word *extern* stands for external storage class.
- The *extern* storage class is used when we have variables or functions which are shared between two or more files.
- Keyword *extern* is used to declare a variable or function in another file to provide the reference of variable or function which have been already defined in the original file.
- The variables defined using an *extern* keyword are called as global variables.
- These variables are accessible throughout the program.
- Notice that an *extern* variable cannot be initialized it has already been defined in the original file.

Exercise

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

Exercise: What will be the output of the following code?

```
#include<iostream>
using namespace std;

 void a(); void b(); void c();
 int x = 1;

  int main(){
  int x = 5;

  cout << "local x in outer scope of main is " << x << endl;
   {
  int x = 7;
  cout << "local x in inner scope of main is " << x << endl;
   }
  cout << "local x in outer scope of main is " << x << endl;
  a();  b();  c();
  a();  b();  c();
  cout << "local x in main is " << x << endl;

  return 0;
  }
```

Exercise (cont.)


```
void a() {
int x = 25;

cout << endl << "local x in a is " << x << " after entering a" << endl;
++x;
cout << "local x in a is " << x << " before exiting a" << endl;
}

void b() {
static int x = 50;
cout << endl << "local static x is " << x << " on entering b" << endl;
++x;
cout << "local static x is " << x << " on exiting b" << endl;
}

void c(){
 cout << endl << "global x is " << x  << " on entering c" << endl;
 x *= 10;
cout << "global x is " << x << " on exiting c" << endl;
}
```

Solution


```
local x in outer scope of main is 5
local x in inner scope of main is 7
local x in outer scope of main is 5

local x in a is 25 after entering a
local x in a is 26 before exiting a

local static x is 50 on entering b
local static x is 51 on exiting b

global x is 1 on entering c
global x is 10 on exiting c

local x in a is 25 after entering a
local x in a is 26 before exiting a

local static x is 51 on entering b
local static x is 52 on exiting b

global x is 10 on entering c
global x is 100 on exiting c
local x in main is 5
```

Conclusions

Chapter 6

Prof Dr. Abbas Khalid

Preface

Inline Functions

Default Arguments

Functions Overloading

Generic Functions

The *const* Qualifier

Storage Classes

Conclusions

In this lecture we discussed about

- Inline functions
- Default arguments
- Functions overloading
- Function template
- *const* return values and parameters
- Storage classes

In the next lecture we will discuss about

- One-dimensional arrays